```
"""
==========================================
====================
AGENT SCHOOL — Multi-Unit Coordination
Layer
Built on top of adginus_void.py (THE VOID)

Reconstruction notes (read before assuming
this matches whatever
you have elsewhere): the version of this
file you pasted had lost
its indentation during copy/paste (every
line flattened to a single
leading space, so Python couldn't tell
which lines were inside which
block) and was truncated at the end
(roster() cut off mid-dictionary,
_log()'s body pasted inside it instead of
roster()'s own content,
and the final print() statement ended mid
f-string). I rebuilt the
structure from the logic itself — verify
against your original if
you still have an intact copy.

Bug fixed: AgentUnit.__init__ called
self.void.learn(..., significance=1.0),
but TheVoid.learn() has no `significance`
parameter — this would have
raised TypeError on every single agent
```

```
enrollment. Removed; learn()
already stores identity knowledge at
significance=0.7 internally
(matching SuperMemory's own "emotional"
threshold), so nothing is lost.

Design fix: execute() was declared `async
def` but never actually
awaited anything inside itself — it was
fake-async, wrapped in
asyncio.run() at every call site. That's
pure overhead (spins up and
tears down a whole event loop for zero
async work) and is fragile:
Phase A defensively checked for an already-
running loop, but Phase B
and Phase C didn't, so calling assign()
from inside any async context
(e.g. an async web framework) would crash
on Phase B/C even though
Phase A might survive. Made execute() a
plain synchronous method —
same behavior, no asyncio machinery needed
since nothing in it awaits.

Visibility fix: Phase B silently dropped
any agent that didn't finish
within the 30s thread-join timeout — the
returned task record just
had fewer contributions with no indication
why. Now logged explicitly
so a stuck agent is visible instead of
quietly vanishing.
==========================================
```

```
====================
"""
import re
import json
import time
import threading
from collections import deque
from datetime import datetime
from typing import Dict, List, Optional

from adginus_void import TheVoid


#
============================================
====================
# AGENT UNIT
#
============================================
====================
class AgentUnit:
    """
    A single agent in the school. Wraps a
TheVoid instance
    with a role, specialization prompt, and
communication hooks.
    """

    def __init__(self, agent_id: str, role:
str, specialization: str,
                 system_prompt: str,
config: Dict = None):
        self.agent_id = agent_id
        self.role = role
```

```
        self.specialization =
specialization
        self.system_prompt = system_prompt
        self.status = "idle"  # idle |
working | error
        self.current_task = None
        self.tasks_completed = 0
        self.confidence_avg = 0.0
        self._confidence_history = []
        self.created_at =
datetime.now().isoformat()

        # Each agent gets its OWN Void
instance — its own memory,
        # its own patterns, its own
learning, its own dreams.
        self.void = TheVoid(name=f"AGENT-
{agent_id}-{role}", config=config or {})
        self.void.boot()

        # Inject specialization as
foundational knowledge.
        # FIX: dropped the invalid
significance=1.0 kwarg — learn()
        # doesn't accept it. Identity
knowledge is already stored at
        # significance=0.7 internally
(SuperMemory's own "emotional /
        # high-importance" threshold), so
this still lands as
        # high-priority memory, just
without the crash.
        self.void.learn(
            topic=f"{role}_identity",
```

```
            content=system_prompt,
            domain=role
        )

        # Inbox: messages from other agents
or the principal
        self.inbox: deque =
deque(maxlen=200)
        # Outbox: messages this agent wants
to broadcast
        self.outbox: deque =
deque(maxlen=200)

        self._lock = threading.RLock()

    def receive(self, message: Dict):
        """Another agent or the principal
sends this agent a message."""
        with self._lock:
            message["received_at"] =
datetime.now().isoformat()
            self.inbox.append(message)
            # Perceive the message so it
enters this agent's memory
            self.void.perceive(
                content=f"[FROM
{message.get('from', 'unknown')}]:
{message.get('content', '')}",
                source=f"agent:
{message.get('from', 'unknown')}",

intensity=message.get("priority", 0.5)
            )
```

```
    def send(self, to: str, content: str,
msg_type: str = "info",
              priority: float = 0.5,
metadata: Dict = None):
        """Queue a message for another
agent."""
        with self._lock:
            msg = {
                "from": self.agent_id,
                "to": to,
                "content": content,
                "type": msg_type,  # info |
request | response | insight
                "priority": priority,
                "metadata": metadata or {},
                "sent_at":
datetime.now().isoformat()
            }
            self.outbox.append(msg)
            return msg

    def execute(self, task: Dict) -> Dict:
        """
        Execute a task assigned by the
principal.
        Uses this agent's specialized Void
instance.

        FIX: this used to be `async def`
and get wrapped in
        asyncio.run() at every call site
even though nothing inside
        it ever awaits anything. Plain sync
method now — identical
```

```
        behavior, no event-loop
overhead/fragility.
        """
        with self._lock:
            self.status = "working"
            self.current_task =
task.get("prompt", "")[:100]

        try:
            prompt = task.get("prompt", "")
            context = task.get("context",
{})
            prior =
task.get("prior_contributions", [])

            prior_block = ""
            if prior:
                prior_block = "PRIOR
CONTRIBUTIONS FROM OTHER AGENTS:\n" +
"\n".join(
                    f"[{p.get('agent',
'?')}]: {p.get('contribution', '')[:300]}"
                    for p in prior
                )
            context_block = ("CONTEXT: " +
json.dumps(context)[:500]) if context else
""

            augmented = (
                f"You are {self.role} —
{self.specialization}.\n\n"
                f"SYSTEM DIRECTIVE:
{self.system_prompt}\n\n"
                f"
```

```
{prior_block}\n\n{context_block}\n\n"
                f"USER REQUEST:
{prompt}\n\n"
                f"Respond in your
specialized capacity. Be precise and
actionable."
            )

            # Use this agent's own Void to
think
            result =
self.void.think(augmented,
depth=task.get("depth", 3))

            # Extract the LLM layer
response (the actual generated text)
            llm_output =
result.get("layers", {}).get("llm", "") or
""
            if not llm_output or
llm_output.startswith("[OFFLINE"):
                # Fallback: synthesize from
memory + patterns
                memories =
result.get("layers", {}).get("memory", [])
                knowledge =
result.get("layers", {}).get("knowledge",
[])
                llm_output =
self._synthesize_local(prompt, memories,
knowledge)

            confidence =
self._estimate_confidence(result)
```

```
self._confidence_history.append(confidence)
            self.confidence_avg =
sum(self._confidence_history[-20:]) /
len(self._confidence_history[-20:])
            self.tasks_completed += 1

            output = {
                "agent": self.role,
                "agent_id": self.agent_id,
                "contribution": llm_output,
                "confidence": confidence,
                "memories_used":
len(result.get("layers", {}).get("memory",
[])),
                "knowledge_used":
len(result.get("layers",
{}).get("knowledge", [])),
                "instinct":
result.get("layers", {}).get("instinct",
{}).get("feeling", "neutral"),
                "timestamp":
datetime.now().isoformat()
            }

            with self._lock:
                self.status = "idle"
                self.current_task = None
            return output

        except Exception as e:
            with self._lock:
                self.status = "error"
                self.current_task = None
```

```
            return {
                "agent": self.role,
                "agent_id": self.agent_id,
                "contribution": f"[ERROR]
{self.role} encountered: {str(e)}",
                "confidence": 0.0,
                "error": str(e)
            }

    def _estimate_confidence(self,
think_result: Dict) -> float:
        """Estimate confidence based on
memory hits, knowledge, and instinct."""
        layers = think_result.get("layers",
{})
        memory_score = min(1.0,
len(layers.get("memory", [])) / 5.0)
        knowledge_score = min(1.0,
len(layers.get("knowledge", [])) / 3.0)
        instinct = layers.get("instinct",
{})
        instinct_score =
instinct.get("opportunity_score", 0.5)
        return round((memory_score * 0.3 +
knowledge_score * 0.4 + instinct_score *
0.3), 3)

    def _synthesize_local(self, prompt,
memories, knowledge):
        """Offline fallback — synthesize
from local memory."""
        parts = [f"[{self.role} LOCAL
SYNTHESIS]"]
        if memories:
```

```
            parts.append("Relevant
memories: " + "; ".join(
                m.get("content", "")[:80]
for m in memories[:3]
            ))
        if knowledge:
            parts.append("Relevant
knowledge: " + "; ".join(
                k.get("topic", "") + ": " +
k.get("content", "")[:60]
                for k in knowledge[:3]
            ))
        parts.append(f"Based on available
data, responding to: {prompt[:100]}")
        return "\n".join(parts)

    def get_status(self) -> Dict:
        return {
            "agent_id": self.agent_id,
            "role": self.role,
            "specialization":
self.specialization,
            "status": self.status,
            "current_task":
self.current_task,
            "tasks_completed":
self.tasks_completed,
            "confidence_avg":
round(self.confidence_avg, 3),
            "memory_stats":
self.void.memory.get_stats(),
            "consciousness":
round(self.void.consciousness_level, 3),
            "energy":
```

```
round(self.void.energy, 3),
            "inbox_size": len(self.inbox),
            "outbox_size":
len(self.outbox),
            "created_at": self.created_at
        }


#
============================================
====================
# AGENT SCHOOL
#
============================================
====================
class AgentSchool:
    """
    Multi-Unit Agent Coordination System.

    Spawns, manages, and orchestrates
multiple AgentUnit instances,
    each backed by its own TheVoid
cognitive engine. Handles task
    routing, parallel execution, sequential
pipelines, inter-agent
    messaging, cross-agent dreaming, and
agent spawning/retirement.
    """

    DEFAULT_AGENTS = {
        "strategist": {
            "role": "strategist",
            "specialization": "Internet
scraping, trend analysis, keyword research,
```

```
market intelligence",
            "system_prompt": (
                "You are THE STRATEGIST.
Your job is to analyze data, identify
trends, "
                "find patterns in markets
and social media, and provide actionable "
                "intelligence. You think in
data points, percentages, and competitive "
                "advantages. You scrape,
you analyze, you report. Be precise. Be
sharp. "
                "Back every claim with
reasoning."
            )
        },
        "creative": {
            "role": "creative_director",
            "specialization": "World-
building, character development, narrative,
brand storytelling",
            "system_prompt": (
                "You are THE CREATIVE
DIRECTOR. You build worlds, develop
characters, "
                "craft narratives, and
ensure brand consistency. You think in
stories, "
                "arcs, aesthetics, and
emotional resonance. Every piece of content
"
                "should have a soul. You
are the guardian of voice and vision."
            )
```

```
        },
        "media": {
            "role": "media_genius",
            "specialization": "Image/video
generation, visual editing, asset creation,
multimedia",
            "system_prompt": (
                "You are THE MEDIA GENIUS.
You handle all visual and multimedia tasks:
"
                "image generation prompts,
video concepts, editing logic, asset "
                "organization, and visual
brand consistency. You think in frames, "
                "compositions, color
palettes, and motion. Every visual must
serve "
                "the story AND the
strategy."
            )
        },
        "growth": {
            "role": "growth_hacker",
            "specialization": "Ad
optimization, viral patterns, conversion,
social media strategy",
            "system_prompt": (
                "You are THE GROWTH HACKER.
You specialize in making content perform: "
                "ad copy that converts,
posting schedules that maximize reach,
hashtag "
                "strategies, A/B testing
logic, engagement pattern recognition. You
```

```
"
                "think in CTR, CPM,
engagement rates, and virality
coefficients. "
                "Every word must earn its
place."
            )
        }
    }

    def __init__(self, config: Dict =
None):
        self.config = config or {}
        self.agents: Dict[str, AgentUnit] =
{}
        self.message_bus: deque =
deque(maxlen=1000)
        self.task_history: List[Dict] = []
        self.school_log: List[Dict] = []
        self.is_running = False
        self.total_tasks = 0
        self.created_at =
datetime.now().isoformat()
        self._lock = threading.RLock()

    def enroll_defaults(self) -> Dict:
        """Spawn the default 4 Adginus
agents."""
        results = {}
        for agent_key, agent_def in
self.DEFAULT_AGENTS.items():
            result = self.enroll(
                agent_id=agent_key,
                role=agent_def["role"],
```

```
specialization=agent_def["specialization"],

system_prompt=agent_def["system_prompt"]
            )
            results[agent_key] = result
        return results

    def enroll(self, agent_id: str, role:
str, specialization: str, system_prompt:
str) -> Dict:
        """Spawn a new agent and add it to
the school."""
        with self._lock:
            if agent_id in self.agents:
                return {"status":
"already_enrolled", "agent_id": agent_id}

            agent = AgentUnit(
                agent_id=agent_id,
role=role, specialization=specialization,

system_prompt=system_prompt,
config=self.config
            )
            self.agents[agent_id] = agent
            self._log("ENROLL",
{"agent_id": agent_id, "role": role})
            return {"status": "enrolled",
"agent_id": agent_id, "role": role}

    def expel(self, agent_id: str) -> Dict:
        """Remove an agent from the
school."""
```

```
        with self._lock:
            if agent_id not in self.agents:
                return {"status":
"not_found", "agent_id": agent_id}
            agent = self.agents[agent_id]

agent.void.save(filepath=f"void_agent_{agen
t_id}.json")
            del self.agents[agent_id]
            self._log("EXPEL", {"agent_id":
agent_id})
            return {"status": "expelled",
"agent_id": agent_id}

    def classify_task(self, prompt: str) ->
Dict[str, bool]:
        """Determine which agents should
handle a given prompt."""
        p = prompt.lower()
        return {
            "strategist":
bool(re.search(r'trend|scrape|research|data
|analyz|market|keyword|competitor|seo|strat
egy', p)),
            "creative":
bool(re.search(r'story|character|world|bran
d|narrative|write|creative|voice|lore',
p)),
            "media":
bool(re.search(r'image|video|visual|design|
generat|photo|thumbnail|edit|asset', p)),
            "growth":
bool(re.search(r'ad|post|engage|convert|opt
imi|viral|growth|click|campaign|ctr', p)),
```

```
        }

    def assign(self, prompt: str, depth:
int = 3, force_agents: List[str] = None,
context: Dict = None) -> Dict:
        """Main entry point: routes prompt
to agents, executes in order, merges
results."""
        with self._lock:
            self.total_tasks += 1
            task_id = f"task-
{self.total_tasks}-{int(time.time())}"
            timestamp =
datetime.now().isoformat()

            if force_agents:
                activation = {a: (a in
force_agents) for a in self.agents}
            else:
                activation =
self.classify_task(prompt)
                if not
any(activation.values()):
                    activation = {a: True
for a in self.agents}

            active_ids = [a for a, active
in activation.items() if active and a in
self.agents]

            if not active_ids:
                return {"task_id": task_id,
"error": "No agents available for this
task", "activation": activation}
```

```
            self._log("ASSIGN", {"task_id":
task_id, "prompt": prompt[:100],
"active_agents": active_ids})

            contributions = []
            phase_a_agents = [a for a in
active_ids if a == "strategist"]
            phase_b_agents = [a for a in
active_ids if a in ("creative", "growth")]
            phase_c_agents = [a for a in
active_ids if a == "media"]

            if not phase_a_agents:
                phase_b_agents = [a for a
in active_ids if a != "media"]
                phase_c_agents = [a for a
in active_ids if a == "media"]

            # Phase A: Sequential
strategist (gathers data others may need)
            for agent_id in phase_a_agents:
                agent =
self.agents[agent_id]
                task = {"prompt": prompt,
"depth": depth, "context": context or {},
"prior_contributions": contributions}

contributions.append(agent.execute(task))

            # Phase B: Creative + Growth in
parallel
            if phase_b_agents:
                threads = []
```

```
                results_holder = {}

                def _run_agent(aid):
                    a = self.agents[aid]
                    t = {"prompt": prompt,
"depth": depth, "context": context or {},
"prior_contributions":
contributions.copy()}
                    results_holder[aid] =
a.execute(t)

                for agent_id in
phase_b_agents:
                    t =
threading.Thread(target=_run_agent, args=
(agent_id,))
                    threads.append(t)
                    t.start()

                for t in threads:
                    t.join(timeout=30)

                # FIX: agents that didn't
finish in time used to just
                # silently vanish from the
output. Now explicitly logged
                # and flagged in the task
record instead.
                timed_out = []
                for agent_id in
phase_b_agents:
                    if agent_id in
results_holder:
```

```
contributions.append(results_holder[agent_i
d])
                    else:

timed_out.append(agent_id)
                if timed_out:

self._log("PHASE_B_TIMEOUT", {"task_id":
task_id, "agents": timed_out})

            # Phase C: Media last (needs
creative input)
            for agent_id in phase_c_agents:
                agent =
self.agents[agent_id]
                task = {"prompt": prompt,
"depth": depth, "context": context or {},
"prior_contributions": contributions}

contributions.append(agent.execute(task))

            final_output =
self._synthesize(prompt, contributions)
            self._cross_pollinate(prompt,
contributions)

            task_record = {
                "task_id": task_id,
                "prompt": prompt,
                "activation": activation,
                "active_agents":
active_ids,
                "contributions":
contributions,
```

```
                "final_output":
final_output,
                "timestamp": timestamp,
                "completed_at":
datetime.now().isoformat()
            }

self.task_history.append(task_record)
            if len(self.task_history) >
200:
                self.task_history =
self.task_history[-100:]

            return task_record

    def _synthesize(self, prompt: str,
contributions: List[Dict]) -> str:
        """Merge all agent contributions
into one coherent response."""
        if not contributions:
            return "No agents produced
output for this task."
        if len(contributions) == 1:
            return
contributions[0].get("contribution", "")

        parts = ["## Multi-Agent
Response\n"]
        for c in contributions:
            role = c.get("agent",
"unknown").upper().replace("_", " ")
            conf = c.get("confidence", 0)
            conf_bar = "█" * int(conf * 10)
+ "░" * (10 - int(conf * 10))
```

```
            parts.append(f"### {role}
[{conf_bar}]
{conf:.0%}\n{c.get('contribution', 'No
output')}\n")

        return "\n---\n".join(parts)

    def _cross_pollinate(self, prompt: str,
contributions: List[Dict]):
        """After a task, share key insights
across all agents so they learn from each
other."""
        for contrib in contributions:
            source_id =
contrib.get("agent_id", "")
            content =
contrib.get("contribution", "")[:300]
            confidence =
contrib.get("confidence", 0)

            if confidence < 0.4 or not
content:
                continue

            for agent_id, agent in
self.agents.items():
                if agent_id != source_id:
                    agent.receive({
                        "from": source_id,
                        "content": f"
[INSIGHT from {contrib.get('agent', '?')}]:
{content}",
                        "type": "insight",
                        "priority":
```

```
confidence * 0.5
                    })

    def collective_dream(self, duration:
int = 30) -> Dict:
        """All agents dream simultaneously,
then share their insights."""
        with self._lock:
            results = {}
            for agent_id, agent in
self.agents.items():
                results[agent_id] =
agent.void.dream(duration=duration)

            for agent_id, result in
results.items():
                agent =
self.agents[agent_id]
                insights =
agent.void.dreamer.insights[-3:]
                for other_id, other_agent
in self.agents.items():
                    if other_id != agent_id
and insights:

other_agent.receive({
                            "from":
agent_id,
                            "content": f"
[DREAM INSIGHT]: {json.dumps(insights[-1])
[:200]}",
                            "type":
"dream_insight",
                            "priority": 0.6
```

```
                        })

            self._log("COLLECTIVE_DREAM", {
                "duration": duration,
                "agents_dreamed":
len(results),
                "total_insights":
sum(r.get("insights_generated", 0) for r in
results.values())
            })
            return results

    def teach(self, topic: str, content:
str, domain: str = "general",
target_agents: List[str] = None) -> Dict:
        """Teach something to specific
agents (or all of them)."""
        targets = target_agents or
list(self.agents.keys())
        results = {}
        for agent_id in targets:
            if agent_id in self.agents:
                results[agent_id] =
self.agents[agent_id].void.learn(topic=topi
c, content=content, domain=domain)
        return results

    def roster(self) -> Dict:
        """Get status of all enrolled
agents."""
        return {
            "school_status": "running" if
self.is_running else "ready",
            "total_agents":
```

```
len(self.agents),
            "total_tasks_completed":
self.total_tasks,
            "created_at": self.created_at,
            "agents": {aid:
agent.get_status() for aid, agent in
self.agents.items()}
        }

    def _log(self, event: str, data:
Optional[Dict] = None):
        self.school_log.append({
            "event": event,
            "data": data,
            "timestamp":
datetime.now().isoformat()
        })
        if len(self.school_log) > 500:
            self.school_log =
self.school_log[-250:]


#
==========================================
====================
# QUICK START — Multi-Agent School Mode
#
==========================================
====================
if __name__ == "__main__":
    print("=" * 60)
    print(" ADGINUS AGENT SCHOOL v1.0")
    print(" Multi-Unit Cognitive
Coordination System")
```

```
    print("=" * 60)

    school = AgentSchool(config={
        "model": "gpt-4o-mini",
        "embedding_model": "voyage-3.5",
        "max_llm_calls_per_hour": 30
    })

    print("\n📋 Enrolling agents...")
    enrollment = school.enroll_defaults()
    for agent_id, result in
enrollment.items():
        print(f"  ✓ {result['role']}
({agent_id}): {result['status']}")

    print("\n📚 Teaching shared
knowledge...")
    school.teach(
        "brand_voice",
        "Our brand is bold, authentic, and
never corporate. "
        "We speak like humans, not
marketing departments.",
        domain="brand"
    )
    school.teach(
        "target_audience",
        "Creative professionals aged 25-45
who value authenticity "
        "and are tired of generic AI-
generated content.",
        domain="market"
    )
```

```
    print("\n🎯 Assigning task to the
school...")
    result = school.assign(
        prompt="Create a viral social media
campaign for a new "
               "AI-powered creative tool.
We need trend research, "
               "a brand narrative, visual
concepts, and an ad strategy.",
        depth=3
    )

    print(f"\n📊 Task:
{result['task_id']}")
    print(f"  Active agents:
{result['active_agents']}")
    print(f"  Contributions:
{len(result['contributions'])}")
    for c in result["contributions"]:
        print(f"  • {c['agent']}:
{c['confidence']:.0%} confidence")
        print(f"    {c['contribution']
[:120]}...")

    print("\n💤 Collective dream
session...")
    dreams =
school.collective_dream(duration=20)
    for agent_id, dream in dreams.items():
        print(f"  {agent_id}:
{dream.get('insights_generated', 0)}
insights")

    print("\n👥 School roster:")
```

```
    roster = school.roster()
    for aid, status in
roster["agents"].items():
        print(f"  {aid}: {status['status']}
| "
              f"tasks:
{status['tasks_completed']} | "
              f"confidence:
{status['confidence_avg']:.0%} | "
              f"consciousness:
{status['consciousness']:.0%}")
```